

Part A — Identify the JOIN Type

Use the **UniTrack** schema from the guide: **Student**, **Department**, **Course**, **Enrollment**.

1 JOIN Type Classification

For each requirement, write which JOIN type is needed (INNER, LEFT, RIGHT, or FULL OUTER):

#	Requirement	JOIN Type
a	Show only students who are enrolled in at least one course	INNER JOIN
b	Show ALL students, including those with no enrollments	LEFT JOIN
c	Show ALL departments, even those with no students	RIGHT JOIN (or LEFT JOIN depend
d	Show ALL students AND ALL departments, matched or not	FULL OUTER JOIN
e	Find students who have NOT enrolled in any course	LEFT JOIN (with WHERE ... IS NULL

Part B — Write JOIN Queries

2 INNER JOIN — Student + Department

Write a query to list every student's Name and their DeptName. Only include students who belong to a department.

```
SELECT S.Name, D.DeptName
FROM Student S JOIN Department D
ON S.DeptID = D.DeptID
```

Expected result: 3 rows (Ahmed, Sara, Omar). Mona is excluded.

3 LEFT JOIN — All Students with Department

Write a query to list **all** students and their department name. Students without a department should show NULL.

```
SELECT S.Name, D.DeptName
FROM Student S LEFT JOIN Department D
ON S.DeptID = D.DeptID
```

Expected result: 4 rows. Mona's DeptName shows NULL.

4 Orphan Finder — Courses with No Enrollments

Write a query using LEFT JOIN + WHERE to find courses that **no student** has enrolled in. Show only CourseID and CourseName.

```
SELECT C.CourseName
FROM Course C JOIN Enrollment E
ON C.CourseID = E.CourseID
WHERE E.CourseID IS NULL
```

Expected result: 1 row (Course 104 — Quantum Physics).

5 3-Table JOIN with Aliases

Write a query to show each student's Name and the CourseName they are enrolled in. Use table aliases (AS). Join: Student → Enrollment → Course.

```
SELECT S.Name, C.CourseName
FROM Students AS S
INNER JOIN Enrollments AS E ON S.StudentID = E.StudentID
INNER JOIN Courses AS C ON E.CourseID = C.CourseID;
```

Expected result: 4 rows. Use S, E, C as aliases.

6 4-Table Full Report

Write a query to produce this report: Student Name, Course Name, Department Name, and Grade. Join all 4 tables, use aliases, and order by student name.

```
SELECT S.Name AS StudentName, C.CourseName, D.DeptName, E.Grade
FROM Students AS S
INNER JOIN Enrollments AS E ON S.StudentID = E.StudentID
INNER JOIN Courses AS C ON E.CourseID = C.CourseID
INNER JOIN Departments AS D ON S.DeptID = D.DeptID
ORDER BY S.Name;
```

Expected result: 4 rows, sorted alphabetically by student name.

Part D — Fix the Broken Query

7 Debug Challenge

The following query has **3 errors**. Find and fix them all:

```
SELECT Student.Name, C.CourseName
FROM Student AS S
INNER JOIN Enrollment
    ON S.StudentID = E.StudentID
LEFT JOIN Course AS C
    ON E.CourseID = C.CourseID
```

Write the corrected query below and explain each fix:

```
SELECT S.Name, C.CourseName
FROM Student AS S
INNER JOIN Enrollment AS E
    ON S.StudentID = E.StudentID
INNER JOIN Course AS C
    ON E.CourseID = C.CourseID;
```

Change: LEFT JOIN Course AS C → INNER JOIN Course AS C

Why: The goal is to show students and the specific courses they are taking. Using an INNER JOIN ensures we only get complete, valid records that exist across all three tables (Student → Enrollment → Course). A LEFT JOIN here is the wrong type

Hints: Look at aliases, missing keywords, and the SELECT clause.

Part E — FULL OUTER JOIN

8 FULL OUTER JOIN Result Prediction

Given the query below, predict the result by filling in the table:

```
SELECT S.Name, D.DeptName
FROM Student AS S
FULL OUTER JOIN Department AS D
ON S.DeptID = D.DeptID;
```

Row	Name	DeptName
1	Ahmed	Computer Science
2	Sara	Mathematics
3	Omar	Computer Science
4	Mona	NULL
5	NULL	Physics

Remember: Mona has NULL DeptID, and the Physics department has no students.

Part F — Bonus Challenge

9 Mixed JOIN Chain

Write a query to show **all** students (even those not enrolled) with their course names. Use LEFT JOIN so Mona appears with NULL for CourseName. Use aliases and sort by Name.

```
SELECT S.Name, C.CourseName
FROM Student AS S
LEFT JOIN Enrollment AS E
ON S.StudentID = E.StudentID
LEFT JOIN Course AS C
ON E.CourseID = C.CourseID
ORDER BY S.Name;
```

Expected result: 5 rows (Ahmed×2, Mona×1 with NULL, Omar×1, Sara×1).

10 Reflection — When to Use Each JOIN

In your own words, explain when you would choose each JOIN type. Give a real-world example for each:

INNER JOIN: You only want records that have a matching pair in both tables.

LEFT JOIN: You want everything from the first table, plus matching info from the second.

RIGHT JOIN: You want everything from the second table, plus matching info from the first.

FULL OUTER JOIN: You want a complete list from both tables, regardless of whether they match.